

구현 소개

1. 서론

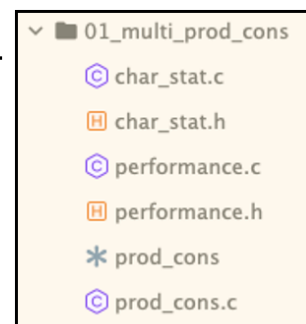
이 프로젝트는 대용량 데이터 처리 시스템에서 발생할 수 있는 Producer-Consumer 구조를 이해하고 발생하는 현상들을 진단하고자 진행되었습니다. 이를 해결하는 다양한 알고리즘을 실험해보자 했습니다. 중국에는 다양한 동기화 알고리즘을 통해 성능을 극대화하는 것을 목표로 합니다. 이를 위해 부품(Component)과 조립(Scenario)의 명확한 분리라는 설계 철학을 바탕으로 모듈화된 아키텍처를 구축하였습니다.

1.1 프로젝트의 구조

가장 많이 공을 들인 부분입니다. 최대한 반복되는 코드를 줄이고 실험을 하기에 최적의 구조를 만들고 싶었습니다.

1.1.1 초기 구조

처음의 구조는 오른쪽과 같이 시나리오가 디렉터리 별로 구분되었습니다. 다음의 구조는 직관적이었지만 문제가 많았습니다. **1 우선 코드 중복 등으로 인해 변경에 대응하기 어려웠습니다.** 스레드 생성, 파일 I/O 관리, 통계 출력 등 공통 로직이 시나리오마다 반복되었습니다. 작은 로직 수정 시에도 모든 파일을 개별적으로 수정해야 했습니다. 변경을 해주지 않으면 제대로 된 성능을 비교할 수 없었습니다.



2 비교 분석이 어려웠습니다. 알고리즘의 핵심 차이점이 중복된 보일러플레이트 코드에 가려질 수 있었습니다. 이 때문에 성능 차이의 원인을 직관적으로 파악하기 어려웠습니다.

1.1.2 리팩토링

Component와 Scenarios를 분리했습니다. 시나리오는 알고리즘을 의미합니다. 개별 알고리즘이 components를 가져다 쓰는 방식으로 구현했습니다.

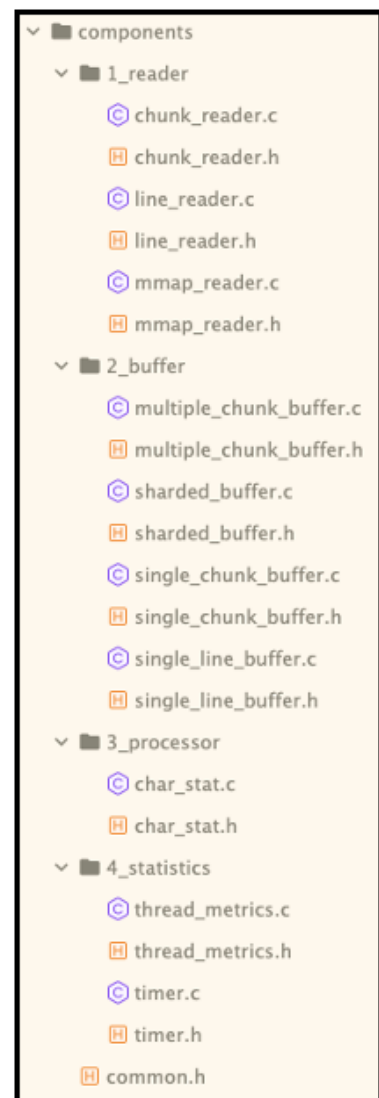
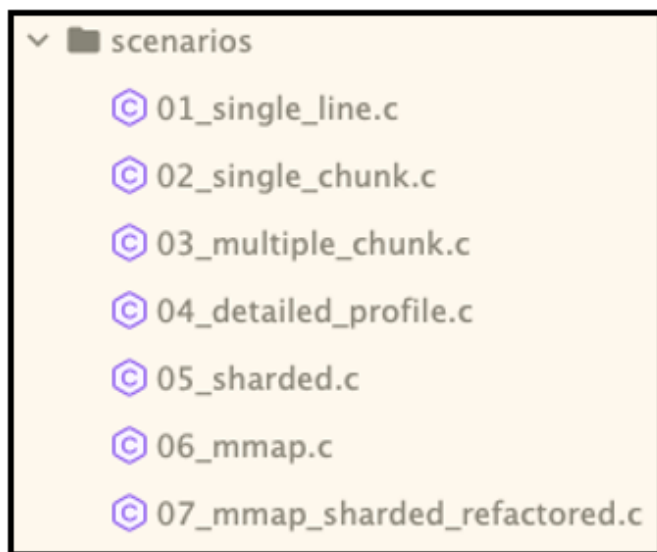
하지만 초기의 방식은 책임이 명확히 분리되지 않았습니다. 버퍼는 데이터 전달과 동기화에만 집중해야 함에도 불구하고 생산자/소비자의 실행 제어까지 관여했습니다. 즉, 결합도가 높아졌습니다. 이는 새로운 Reader나 Processor를 결합할 때 버퍼 내부 코드를 수정해야 했습니다. components를 분리한 의미를 없앴습니다.

1.1.3 최종 구조

책임을 명확히 분리하여 다음과 같이 Component와 Scenarios를 분리했습니다.

Components는 4가지로 구분됩니다. 각각은 다음과 같습니다.

- 1_reader: 데이터 소스 접근 전략 (Line, Chunk, Mmap 방식 분리)
- 2_buffer: 생산자-소비자 간 데이터 교환 메커니즘 (Single, Multiple, Sharded 구조)
- 3_processor: 데이터 처리 알고리즘 (Zero-Copy 기반 알파벳 통계 분석)
- 4_statistics: 성능 측정 및 리포팅 도구 (Timer, Metrics 통계)



2. 시나리오

2.1 single_line

1. 소개

이 시나리오는 Producer-Consumer 패턴의 가장 기초적인 형태를 구현한 것입니다. 텍스트 파일을 처리하는 단위를 한 줄(Line)로 설정했습니다. Producer는 파일에서 한 줄을 읽어 버퍼에 넣습니다. Consumer는 버퍼에서 한 줄을 꺼내 통계를 처리합니다. 이때 버퍼는 오직 하나의 데이터만 담을 수 있는 단일 슬롯입니다. 이는 성능 최적화의 Baseline이 되는 모델로입니다.

2. components의 역할

- **LineReader**: `getline()`을 사용하여 파일 입출력을 수행합니다. 파일 포인터를 관리하며 지정된 범위(`ReadRange`) 내의 데이터만 읽도록 제어합니다.
- **SingleLineBuffer**: Producer와 Consumer 사이의 데이터 통로입니다. 크기가 1인 공간을 가집니다. `Mutex`와 `Condition Variable`를 사용해 스레드 간의 순서를 조율합니다.

3. 구현시 신경 쓴 부분

- **메모리 누수 관리**: `LineReader`가 할당한 메모리의 포인터가 버퍼를 거쳐 Consumer에게 전달됩니다. 따라서 메모리 해제의 책임이 Consumer에게 있음을 명확히 하여 `Memory Leak`을 방지했습니다.
- **종료**: 여러 Producer가 존재할 때, 모든 Producer가 작업을 마쳐야만 Consumer가 종료되도록 설계했습니다. 이를 위해 버퍼 내부에서 활성 Producer의 수(`active_producer`)를 카운팅하여, 마지막 Producer가 종료될 때 Consumer들에게 종료 신호를 보내도록 구현했습니다.

2.2 single_chunk

1. 소개

이 시나리오는 파일 입출력(I/O)의 효율성을 높이기 위해 고안되었습니다. 이전 시나리오에서는 `\n`을 찾으며 한 줄씩 읽었습니다. 여기서는 고정된 크기의 `Chunk` 단위로 데이터를 읽습니다. Producer는 지정된 크기만큼 데이터를 읽어 버퍼에 넣습니다. Consumer는 덩어리째로 데이터를 받아 통계를 처리합니다. 버퍼 구조는 여전히 단일 슬롯 방식을 유지했습니다. 이를 통해, I/O 방식 변경에 따른 순수한 성능 변화를 관찰할 수 있도록 설계했습니다.

2. components의 역할

- **ChunkReader**: `pread`시스템 콜을 사용하여 파일을 읽습니다. 파일 포인터를 이동시키지 않습니다. 대신 `Offset`을 직접 관리하며 스레드 안전하게 데이터를 읽어옵니다. 매 호출마다 `Chunk` 크기의 메모리를 할당합니다.

- SingleChunkBuffer: DataUnit 구조체를 하나 저장할 수 있는 공유 버퍼입니다. SingleLineBuffer와 동기화 로직은 동일합니다. 다루는 데이터 타입만 DataUnit 구조체로 변경되었습니다.

3. 구현시 신경 쓴 부분

- DataUnit 구조체 도입: 단순 char* 대신 DataUnit 구조체를 도입했습니다. 바이너리 데이터나 덩어리 데이터를 다룰 때는 데이터의 시작 주소뿐만 아니라 크기 정보도 필수적이기 때문입니다. 이를 통해 버퍼가 데이터의 구체적인 내용을 몰라도 전달할 수 있게 추상화했습니다
- pread의 활용: fread나 fgets 대신 pread를 사용했습니다. 이는 파일 디스크립터의 오프셋을 변경하지 않고 읽기 때문입니다. 이 방식은 향후 멀티 스레드가 동시에 파일을 읽는 병렬 구조로 확장할 때 Race Condition을 피할 수 있게 해줍니다.

2.3 multiple_chunk

1. 소개

이 시나리오는 Producer와 Consumer 사이의 속도 차이를 완충하기 위해 Multiple Slot Buffer를 도입한 모델입니다. 이전의 단일 슬롯 버퍼는 데이터를 하나 넣으면 소비될 때까지 멈춰야 했습니다. 이 모델은 버퍼에 빈 공간이 있다면 Producer가 멈추지 않고 계속 데이터를 생산할 수 있습니다. 반대로 버퍼에 데이터가 쌓여 있다면 Consumer도 Producer를 기다리지 않고 연속적으로 데이터를 소비할 수 있습니다. 이를 통해 스레드 간의 결합도를 낮추고 전체적인 Throughput을 향상시키는 것이 목표입니다.

2. components의 역할

- MultiChunkBuffer: 핵심 컴포넌트로, 고정된 크기의 배열을 가진 원형 큐입니다. 내부적으로 head(읽기), tail(쓰기), count(갯수)를 관리합니다. FIFO 방식으로 데이터를 전달합니다.

3. 구현시 신경 쓴 부분

- 원형 큐(Circular Queue) 자료구조: 배열의 끝에 도달하면 다시 처음으로 돌아가는 원형 구조를 사용하여, 메모리 재할당이나 데이터 이동 없이 효율적으로 큐를 구현했습니다.
- 동적인 버퍼 크기 조절: 프로그램 실행 시 커맨드 입력을 통해 큐의 크기를 조절할 수 있습니다. 이를 통해 버퍼 크기가 성능에 미치는 영향을 실험적으로 관찰할 수 있게 했습니다.

2.4 profile

1. 소개

이전 시나리오들에서는 단순히 총 실행 시간만을 측정했습니다. 하지만 총 시간만으로는 시스템이 왜 느린지 정확히 알 수 없습니다. 디스크가 느린 것인지, 계산이 복잡한 것인지, 아니면 스레드끼리 contention하는지 알 수 없기 때문입니다.

이 시나리오는 가장 빨랐던 이전 시나리오를 대상으로 각 시간을 정밀하게 측정했습니다. 다시 말해 스레드의 Lifecycle를 정밀하게 추적했습니다. 각 스레드가 I/O, 대기, 연산에 각각 얼마의 시간을 소비하는지 ns 단위로 측정했습니다. 이를 통해 시스템의 진짜 병목이 어디에 있는지 수치로 증명하고자 했습니다. 이를 근거로 다음 단계의 최적화 전략을 수립하는 것이 이 시나리오의 목적입니다.

2. 측정 내용

- I/O Time

디스크에서 데이터를 읽어오는 순수한 비용을 알기 위해 측정했습니다. 하드웨어 디스크가 병목이거나 시스템 콜(read/pread) 오버헤드를 판단하고자 했습니다.

chunk_reader_next() 함수 호출 전후로 tm_start와 tm_stop을 사용하여 측정했습니다.

- Wait/Locking Time

이 시간은 유용한 작업을 하는 시간이 아닙니다. 락을 얻기 위해 줄을 서서 기다리는 낭비된 시간입니다. lock contention이 얼마나 심각한지 알고자 측정했습니다.

버퍼에 데이터를 넣거나 꺼내는 함수 호출 시간을 측정했습니다.

- Compute Time

순수하게 데이터를 파싱하고 통계를 내는 CPU 연산 비용입니다. 이번 주제와 큰 관련은 없지만 전체적인 성능 향상에 있어서는 중요한 포인트라고 생각했습니다.

- Page Faults & Context Switches

Major Page Fault를 통해 디스크 I/O가 얼마나 발생했는지 확인하고자 했습니다.

스레드가 락을 얻지 못해 자발적으로 CPU를 양보한 경우가 얼마나 많은지 측정하고자 했습니다. 이 수치 또한 높으면 락 경합이 매우 심하다는 뜻입니다.

리눅스 시스템 콜 getrusage(RUSAGE_THREAD)를 사용했습니다.

2.5 sharded_chunk

1. 소개

이전 단계인 03_multiple_chunk에서는 원형 큐를 도입하여 I/O 대기 시간을 줄였습니다. 하지만 스레드 수가 증가함에 따라 Lock Contention이라는 새로운 병목이 발생했습니다. 다수의 스레드가 단 하나의 락을 차지하기 위해 줄을 섰습니다. 이 때문에 실제 일하는 시간보다 대기하는 시간이 길어지는 현상이 발생했습니다.

이번 시나리오는 이러한 락 경합 문제를 구조적으로 해결하기 위해 설계되었습니다. 핵심 아이디어는 여러 개의 버퍼를 만드는 것입니다. 하나의 거대한 공유 버퍼를 사용하는 대신 버퍼를 여러 개의 독립적인 Shard로 분할합니다. Producer는 자신에게 배정된 전용 샤드에만 데이터를 넣게 됩니다. 이를 통해 쓰기 경합을 원천적으로 차단합니다. 반면, Consumer는 자신의 샤드가 비었을 때 다른 샤드의 데이터를 가져오는 유연한 구조를 가집니다. 이를 통해 동기화 비용을 최소화하면서도 CPU 자원을 100% 활용하는 고성능 병렬 처리를 구현하고자 했습니다.

2. components의 역할

- **ShardedBuffer**: 여러 개의 **MultiChunkBuffer**를 관리하는 상위 컨테이너입니다. 내부적으로 다수의 하위 버퍼 포인터 배열을 가집니다. 전체 시스템의 종료 상태와 동작 중인 Producer 수를 관리하기 위한 별도의 글로벌 락을 관리합니다.
- **MultiChunkBuffer (Shard)**: 실제 데이터가 저장되는 하위 버퍼입니다. 이 시나리오에서는 기존의 **get/put** 기능 외에 락을 기다리지 않고 즉시 상태를 반환하는 **try_get** 기능이 추가되어 **Work Stealing**을 지원합니다.

3. 구현시 신경 쓴 부분

- **Lock Contention 없애기**: Producer들은 서로 완벽하게 격리됩니다. 각자 전용 라인(Shard)을 사용하므로, Producer 간의 락 충돌이 없어집니다. 이는 I/O 속도가 허용하는 한 무한한 확장성을 보장합니다.
- **Work Stealing**: 특정 파일 구간에 데이터가 많거나 특정 Consumer가 느려지더라도 전체 성능에 영향을 주지 않습니다. **Work Stealing** 알고리즘을 추가하여 바쁜 샤드의 데이터는 한가한 Consumer들에 의해 자동으로 분산 처리됩니다.
- **재사용**: **ShardedBuffer**는 기존에 검증된 **MultiChunkBuffer** 코드를 사용합니다. 또한 샤드의 개수를 실행 인자로 받아 Producer 수와 다르게 설정해보았습니다. 최적의 성능 지점을 찾을 수 있도록 유연하게 설계했습니다.
- **변경**: 기존 **pthread_mutex_lock**은 락을 얻을 때까지 스레드를 재워버립니다. 그렇기 때문에 **Work Stealing**에 적합하지 않았습니다. **trylock**을 도입하여 스레드가 멈추지 않고 계속해서 유효한 작업을 찾아다니도록 흐름을 제어했습니다.

4. 동작 설명

a. 탐색 (Work Stealing)

먼저 전체 샤드 리스트를 순회하며 **pthread_mutex_trylock**을 시도합니다.

b. 종료 조건의 이중 확인

- 모든 샤드를 찢어봤는데도 데이터가 없다면, 전체 시스템이 종료되었는지 확인합니다.

- 이때 단순히 플래그(finished)만 보는 것이 아니라, 실제로 모든 샤드의 데이터 개수(count)가 0인지 다시 한번 확인하여 데이터가 남았는데 종료되는 오류를 방지합니다.

c. 대기

- 가져올 데이터도 없고 시스템이 끝난 것도 아니라면 자신의 전용 샤드로 돌아갑니다. 이후 `pthread_cond_wait`로 대기합니다. 이를 통해 busy waiting을 방지합니다.

2.6 mmap

1. 소개

이번 시나리오는 이러한 데이터 복사 비용과 시스템 콜 오버헤드를 제거하기 위해 Memory Mapped I/O 기법을 도입했습니다. 이전 시나리오들에서 사용한 `read`나 `pread` 시스템 콜은 디스크의 데이터를 커널 버퍼로 읽어 들입니다. 이후 다시 사용자 영역의 버퍼로 복사하는 과정이 필요합니다. 하지만 이번 시나리오에서는 파일을 프로세스의 가상 메모리 주소 공간에 직접 매핑합니다. 이를 통해 마치 메모리에 있는 배열을 읽는 것처럼 구현하고자 했습니다. 사용자 영역에서의 버퍼 할당과 데이터 복사 과정을 생략하고 운영체제의 페이지 캐시 기능을 적극적으로 활용하여 성능을 높이고자 했습니다.

2. components의 역할

- **MmapReader**: mmap 시스템 콜을 사용하여 파일을 메모리에 매핑합니다. 파일의 특정 오프셋에 해당하는 메모리 주소를 계산하여 반환합니다. 일반적인 Reader와 달리 실제로 데이터를 읽거나 복사하지 않습니다. 대신 데이터가 위치한 주소만을 제공합니다.
- **Producer Thread**: MmapReader를 통해 파일 데이터가 매핑된 메모리 주소를 획득합니다. 이 주소를 `DataUnit`에 담아 버퍼에 전달합니다. 중요한 점은 데이터를 힙에 할당하지 않습니다. 대신 매핑된 영역의 포인터를 그대로 전달합니다.

3. 구현시 신경 쓴 부분

- **Page Alignment**: mmap 시스템 콜은 반드시 페이지 크기의 배수 위치에서만 매핑을 시작할 수 있습니다. 이를 위해 `range.start`를 페이지 크기에 맞춰 비트 연산으로 내림 처리합니다. 오차 범위를 보정하여 정확한 데이터 위치를 반환하도록 구현했습니다.
- **메모리 생명주기 관리**: 일반적인 Reader와 달리 mmap은 Producer가 읽기를 마쳤다고 해서 바로 해제하면 안 됩니다. Consumer가 아직 큐에 남아있는 포인터를 참조하고 있을 수 있기 때문입니다. 따라서 Producer 스레드 내부가 아닌 모든 작

업이 완료된 후 Main 함수에서 reader를 정리합니다. 이러한 구조로 Segmentation Fault를 방지했습니다.

- PROT_WRITE: 통계 처리 로직이 원본 문자열을 수정합니다. 이 때문에 읽기 전용으로 매핑해서 크래시가 발생했었습니다. 이를 방지하기 위해 PROT_WRITE 권한을 추가하고 MAP_PRIVATE 모드를 사용했습니다. 데이터 수정 시 원본 파일은 건드리지 않고 해당 페이지만 복사본을 생성하도록 처리했습니다.

4. mmap을 사용하는 것은 producer-consumer 방식의 해결책이 될 수 있는가?

제가 구현하면서 가장 고민을 많이 한 부분입니다. read를 사용하는 방식은 버퍼에 데이터를 올립니다. 그리고 이 데이터를 consumer는 소비합니다.

반면 mmap을 사용한 방식은 주소를 전달합니다. mmap은 copy를 하지 않습니다. mmap은 커널 버퍼의 메모리 주소를 프로세스의 주소 공간에 매칭합니다. 이 주소를 consumer에 넘기면 consumer가 이에 접근하여 소비합니다. 버퍼로 데이터를 넘기지 않기에 이것이 p-c 문제의 해결책이 아닐 수도 있다고 생각했습니다. 결론적으로 kernel의 페이지 캐시를 공유 버퍼로 사용합니다. 그렇기에 잘 모르겠습니다.

3. 성능 측정

- 44GB 크기의 파일로 측정했습니다.
- 실험 환경
 - cpu model: Intel(R) Xeon(R) Gold 5320
 - 52 threads / 26 cores
 - L1 / L2 / L3 : 1.2 / 32.5 / 39

3.1 전체 비교

다음은 제가 측정한 시나리오들입니다.

- 3번 시나리오는 제가 baseline으로 잡은 시나리오입니다. 원형큐로 버퍼를 만들었습니다.
- 5번 시나리오는 락을 분리하여 여러 버퍼를 갖도록 한 시나리오입니다. 개별 버퍼는 3번 시나리오의 버퍼를 사용합니다.
- 7번 시나리오는 mmap을 사용한 시나리오입니다. 버퍼에는 포인터를 넘겨 copy를 줄인 방식입니다.
- 이것들에 대한 정리는 /resultsOfScenarios/final_performance_comparison.log에 있습니다.

지표	05_sharded	07_mmap	03_multi_chunk
시간	02.74	04.25	20.81

3.2 전반부 비교

지표 16:16	01_single_line	02_single_chunk	03_multiple_chunk
Elapsed Time	48.06s	3.19s	0.49s
System Time	341.83s	17.24s	9.01s
CPU Usage	843%	883%	2476%
Context Switches	19,628,947	1,287,949	429,702

스레드 설정 (P:C)	01_single_line	02_single_chunk	03_multiple_chunk
1:1	21.09s	2.07s	2.19s
4:4	56.16s	1.55s	0.85s
16:16	48.06s	3.19s	0.49s

1. 01_single_line

- 천만 단위의 Context Switch: 16:16 환경에서 약 1,960만 번의 문맥 교환이 발생했습니다. 이는 아주 작은 데이터를 처리할 때마다 Lock Contention이 발생하여 스레드가 잠들고 깨어나는 데 모든 시간을 허비했음을 의미합니다.
- 비정상적인 System Time: 실행 시간은 48초인데 커널 시간(System Time)이 341초입니다. 이는 CPU가 실제 일을 하기보다 OS가 스레드를 관리하고 락을 처리하는 데 과부하가 걸렸음을 뜻합니다.

2. 02_single_chunk

- 데이터 배치 처리: 처리 단위를 Chunk(4KB)로 늘리자마자 1:1 기준 21초에서 2초로 성능이 10배 점프했습니다. 시스템 콜 횟수가 줄어들며 System Time이 급격히 안정화되었습니다.
- 확장성의 한계: 하지만 스레드가 4:4에서 16:16으로 늘어날 때 실행 시간이 오히려 1.55s에서 3.19s로 나빠졌습니다. 이는 버퍼 슬롯이 1개뿐이라 스레드가 많아질수록 락 경쟁이 다시 심해졌기 때문입니다.

3. 03_multiple_chunk

- 완벽한 확장성: 16:16 환경에서 2476%의 CPU 사용률을 기록했습니다. 이는 24개 이상의 코어를 동시에 풀가동했다는 의미입니다. 100개의 버퍼 슬롯이 생산자와 소비자 간의 간섭이 완화되었다는 것을 의미합니다.

- 압도적 효율: 스레드가 늘어날수록 성능이 선형적으로 향상되었습니다.

3.3 baseline 분석

저는 03_multiple_chunk를 baseline으로 두고 작업을 했습니다. 이것보다 빠른 알고리즘들을 만드는 것을 목표로 했습니다. 그래서 이 시나리오의 문제점을 분석하고자 했습니다. 어떤 부분을 발전시켜야 성능이 오를지 분석했습니다. 분석을 위해 4_detailed_profile을 작성했습니다.

```

===== [ Detailed Performance Analysis ] =====
[Producer Statistics]
ID | I/O Time(s) | Put/Wait(s) | MinFault (RAM) | MajFault (Disk) | CtxSwitch
-----
0 | 1.1263 | 18.8559 | 299 | 0 | 713863
1 | 1.1284 | 18.8998 | 32 | 0 | 789609
2 | 1.1156 | 18.8437 | 178 | 0 | 711803
3 | 1.0701 | 18.7818 | 173 | 0 | 718946
4 | 1.0960 | 18.8786 | 232 | 0 | 714042
5 | 1.1084 | 18.8773 | 142 | 0 | 713727
6 | 1.1029 | 18.8488 | 128 | 0 | 718932
7 | 1.1069 | 18.8728 | 376 | 0 | 718166
8 | 1.0862 | 18.8455 | 227 | 0 | 713658
9 | 1.1091 | 18.8841 | 93 | 0 | 713438
10 | 1.1035 | 18.8875 | 187 | 0 | 712133
11 | 1.0822 | 18.8169 | 131 | 0 | 714697
12 | 1.1007 | 18.8768 | 32 | 0 | 789725
13 | 1.1217 | 18.9861 | 125 | 0 | 718114
14 | 1.0947 | 18.8765 | 27 | 0 | 713946
15 | 1.0998 | 18.8796 | 51 | 0 | 718247
  
```

```

[Consumer Statistics]
ID | Get/Wait(s) | Process(s) | Chunks | MinFault | CtxSwitch
-----
0 | 13.6427 | 6.2536 | 711563 | 2 | 453828
1 | 13.7833 | 6.1161 | 715874 | 2 | 464897
2 | 13.7900 | 6.0991 | 717875 | 2 | 464814
3 | 13.7610 | 6.1292 | 714570 | 2 | 463546
4 | 13.7605 | 6.1473 | 713316 | 2 | 459118
5 | 13.6232 | 6.2651 | 701305 | 2 | 453387
6 | 13.7079 | 6.1827 | 709649 | 2 | 459689
7 | 13.7636 | 6.1229 | 714097 | 2 | 462868
8 | 13.7507 | 6.1346 | 716063 | 2 | 462769
9 | 13.6581 | 6.2288 | 705545 | 2 | 457286
10 | 13.6511 | 6.2432 | 702536 | 2 | 454601
11 | 13.7237 | 6.1637 | 713606 | 2 | 460337
12 | 13.5326 | 6.1627 | 698398 | 2 | 449982
13 | 13.7249 | 6.1627 | 713959 | 2 | 462638
14 | 13.7536 | 6.1376 | 716084 | 2 | 462984
15 | 13.6921 | 6.1935 | 709656 | 2 | 458316

[Bottleneck Diagnosis]
1. Avg I/O Time : 1.1033 sec (Producer Disk Read)
2. Avg Locking Time : 16.2828 sec (Prod Wait: 18.8593, Cons Wait: 13.7062)
3. Avg Compute Time : 6.1839 sec (Consumer Processing)
  
```

3.3.1 10GB 파일

지표 10GB	1P : 1C	4P : 4C	16P : 16C
Total Time (Real)	21.25s	8.77s	4.66s
Avg Locking Time	10.06s	5.54s	3.75s
Avg Compute Time	18.22s	5.14s	1.46s
CPU Sys	7.37s	14.86s	83.10s
CPU User	22.48s	28.06s	30.82s
Context Switches(P)	252만	318만	267만
Context Switches(C)	228	110만	172만

3.3.2 44GB 파일

지표 44GB	1P : 1C	4P : 4C	16P : 16C
Total Time (Real)	91.69s	33.80s	19.74s
Avg Locking Time	43.48s	21.09s	15.92s
Avg Compute Time	78.73s	20.41s	6.19s
CPU Sys	31.98s	57.97s	358.68s

CPU User	96.71s	110.98s	130.78s
Context Switches(P)	110만	1426만	1140만
Context Switches(C)	894	476만	740만

10GB와 44GB 두 사례 모두 스레드 수가 증가함에 따라 전체 실행 시간(Real Time)은 단축되었습니다. 10GB 파일은 1:1 환경의 21.25초에서 16:16 환경의 4.66초로 약 4.5배 빨라졌습니다. 44GB 파일 역시 1:1 환경의 91.68초에서 16:16 환경의 19.74초로 약 4.6배의 성능 향상을 보였습니다. 하지만 스레드 수 증가에 비례하여 커널 부하와 동기화 비용이 급격히 상승하는 공통적인 한계가 관측되었습니다.

데이터 규모와 상관없이 스레드가 많아질수록 실제 연산 시간보다 락을 기다리는 시간이 더 길어졌습니다.

- 10GB 사례 (16:16): 평균 연산 시간(Compute)은 1.46초인 반면, 락 대기 시간(Locking)은 3.75초로 약 2.5배 더 깁니다.
- 44GB 사례 (16:16): 평균 연산 시간은 6.19초이나, 락 대기 시간은 15.92초로 역시 연산 시간의 2.5배를 상회합니다.

스레드 수가 증가할수록 운영체제 커널이 소비하는 시간이 폭발적으로 증가했습니다.

- 10GB 사례: 16:16 환경에서 Sys Time은 83.10초로, User Time(30.82초)의 약 2.7배에 달합니다.
- 44GB 사례: 16:16 환경에서 Sys Time은 358.68초로, User Time(130.78초)보다 약 2.7배 높았습니다.

Major Page Faults: 모든 측정에서 0을 기록했습니다. 이는 데이터가 이미 OS의 페이지 캐시에 로드되어 있음을 의미합니다. 따라서 현재의 병목은 물리적 디스크 속도가 아닌 순수한 소프트웨어 구조의 문제입니다.

Voluntary Context Switches: 10GB 1:1 환경에서 250만 번, 44GB 1:1 환경에서 1,100만 번 이상 발생했습니다. 락을 얻지 못해 스레드가 잠들고 깨어나는 오버헤드가 CPU 자원을 심각하게 낭비하고 있습니다.

3.3.3 sharded vs multiple_chunk

지표 (Elapsed Time)	P:1 / C:1	P:8 / C:8	P:16 / C:16
Scenario 03 (Single Lock)	1:41.01	26.63s	20.74s
Scenario 05 (Sharding)	1:31.06	11.96s	7.50s

지표 P:16 / C:16	Single Lock (기존)	Sharding (개선)
----------------	------------------	---------------

Elapsed Time (소요 시간)	20.74초	7.50초
System Time (커널 시간)	369.74초	43.11초
User Time (연산 시간)	132.00초	110.20초
Context Switches	1,935만	966만

- **Single Lock의 경우 sys time이 369초**입니다. 20초 동안 실행되면서 CPU 시간 370초를 락 관리에 태워버렸습니다.

Sharding의 경우 sys time이 43초입니다. 스레드가 32개(16+16)로 늘어났음에도 오버헤드가 크게 늘지 않았습니다.

Sharding 모델은 생산자(Producer) 격리를 통해 불필요한 경쟁을 없앨 수 있었습니다. 이로 인해 컨텍스트 스위칭 횟수가 절반으로 줄었습니다.

3.3.4 최적화

1. sharded

Producer (P)	Consumer (C)	Buffer Size	Elapsed Time (sec)
8	16	100	2.90
8	16	1000	2.88
8	32	100	5.46
8	32	1000	5.31
8	64	100	7.68
8	64	1000	7.97
8	100	100	7.70
8	100	1000	7.58
16	16	100	3.46
16	16	1000	3.49
16	32	100	3.10
16	32	1000	3.10
16	64	100	4.38
16	64	1000	4.12
16	100	100	5.82
16	100	1000	5.75
32	32	100	3.57
32	32	1000	3.63

32	64	100	3.46
32	64	1000	3.80
32	100	100	3.32
32	100	1000	3.28
100	100	100	3.87
100	100	1000	4.41

2. mmap

Producer (P)	Consumer (C)	Buffer Size	Elapsed Time (sec)
8	16	100	6.82
8	16	1000	6.71
8	32	100	5.19
8	32	1000	4.97
8	64	100	4.41
8	64	1000	4.24
8	100	100	4.46
8	100	1000	4.29
16	16	100	7.99
16	16	1000	8.36
16	32	100	5.21
16	32	1000	5.18
16	64	100	4.72
16	64	1000	4.72
16	100	100	4.76
16	100	1000	4.51
32	32	100	5.67
32	32	1000	5.46
32	64	100	5.13
32	64	1000	5.23
32	100	100	4.88
32	100	1000	5.03
100	100	100	5.54

100	100	1000	5.54
-----	-----	------	------

4. 실행 방법

시나리오는 최소 3개, 최대 5개의 인자를 받습니다.

파일 / producer의 개수 / consumer의 개수 / buffer 크기 / shard 개수

```
// 시나리오 1
./01_single_line large_random_file.bin 10 10
// 시나리오 2
./02_single_chunk large_random_file.bin 10 10
// 시나리오 3
./03_multiple_chunk large_random_file.bin 10 10 20
// 시나리오 4
./04_detailed_profile large_random_file.bin 10 10 20
// 시나리오 5
./05_sharded large_random_file.bin 10 10 10 10
// 시나리오 6
./06_mmap large_random_file.bin 10 10 100
// 시나리오 7
./07_mmap_sharded large_random_file.bin 10 10 10 10
```